Operating Systems Technologies

(Tecnologias de Sistemas Operativos)

Benchmarking

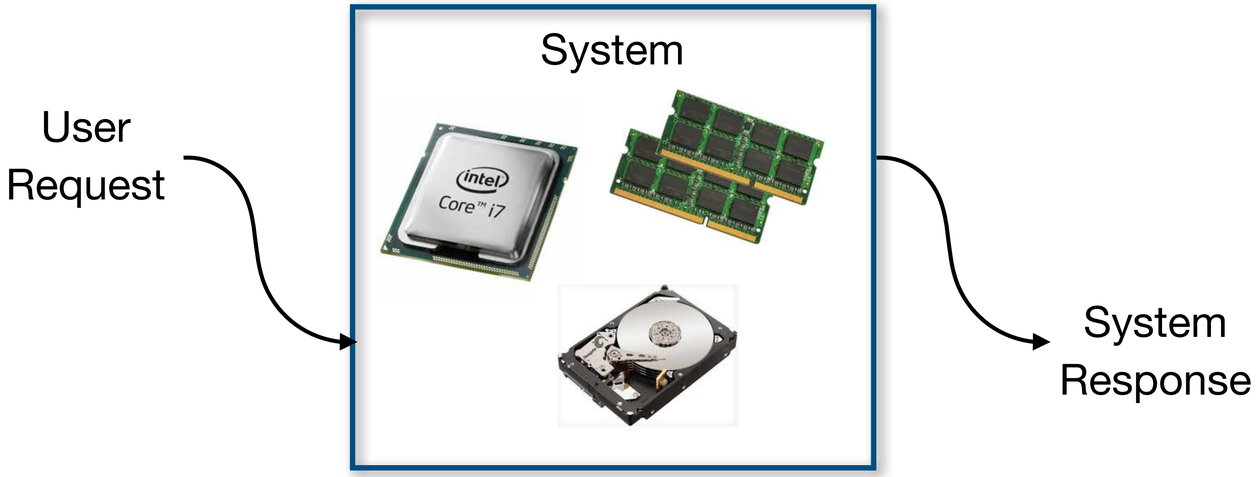
University of Minho

2025-2026

Software Systems

What do these do?

- Software systems can assume different forms (e.g., operating system (OS), file system, database, web server, …)
- Systems perform useful work for users by receiving **requests**, handling these, and producing **responses**
- Systems use physical and logical resources with limited **capacity**
  - **Physical:** CPU, memory, disk, network, …
  - **Logical:** locks, caches, …



Systems Benchmarking

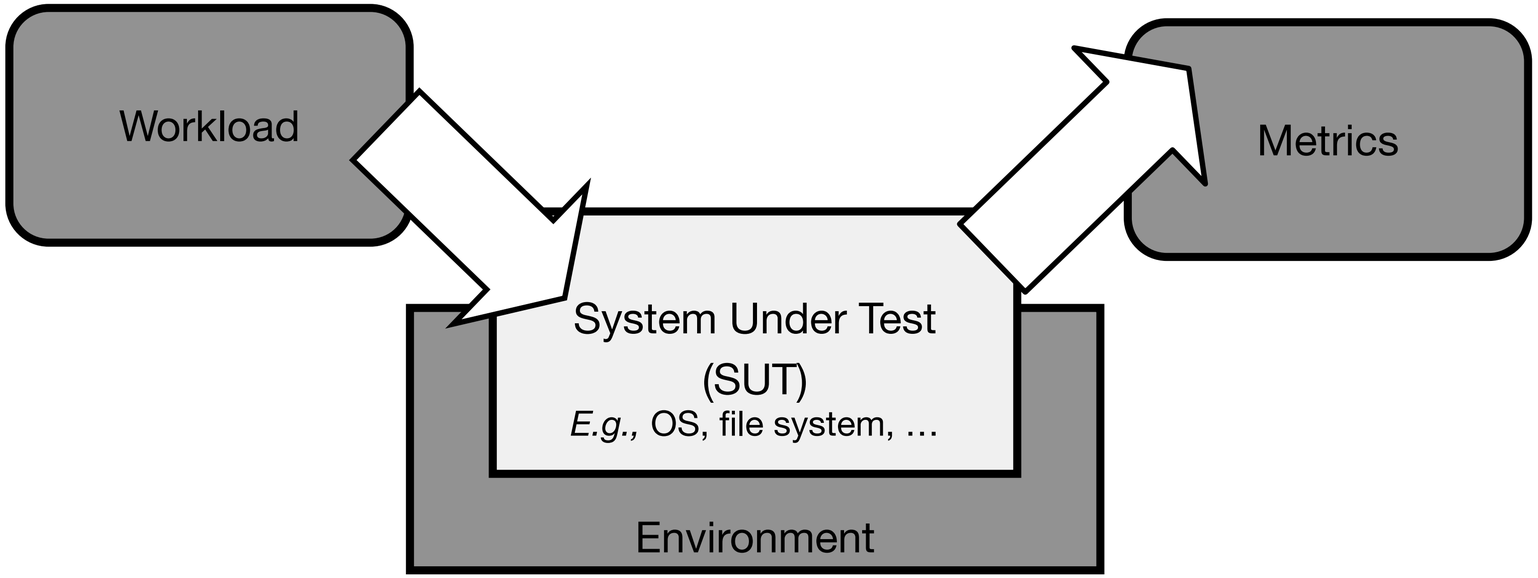
Why is it useful?

- How can one build performant and reliable systems?
  - Follow good design and implementation guidelines
  - Review and validate the code and implementation (e.g., formal verification)
  - **Test** (i.e., **benchmark**) implementations!
- How can one know that the system meets the desired requirements?
  - **Benchmark** the system!
- How can one know the system is using logical and physical resources efficiently?
  - **Benchmark** the system!
- How can one know that the environment where the system is running has enough resources?
  - **Benchmark** the system in that environment!

Systems Benchmarking

Ecosystem

- A Benchmarking ecosystem consists of
  - The **workload** (group of requests) being issued to the **System Under Test** (SUT)
  - The **environment** where the SUT is running
  - The **metrics** collected to measure the performance, efficiency, and/or reliability of the SUT



Workload

Realistic workloads

- Testing systems in **production** may not be viable, or the best option
  - Imagine only knowing whether your OS scheduler works or whether the file system actually stores user data, after deployment and while serving users…
- One option is to use **traces of requests** from real workloads (e*.g.*, from users or applications using the OS)
  - Finding traces and scaling these without losing realism is not easy… (i.e., How can one scale a trace with 100 requests to millions of requests without losing realism?)
  - Take a look at http://iotta.snia.org/traces and https://traces.cs.umass.edu/ for some examples of memory, system call, network, and block I/O traces
- Another option is to use **real applications**
  - E.g., run a database (MySQL, Postgres) or an AI framework (PyTorch, TensorFlow) over the OS
  - We still need to find the workloads to run over these applications to make their interaction with the OS realistic…

Workload

Synthetic workloads

- Yet another option is to use **synthetic workloads**, which are useful
  - When the aforementioned options are not viable
  - To evaluate specific functionalities of the SUT (*e.g.,* stress the CPU, memory, storage, …)
  - To stress the SUT and/or test its scalability (*i.e.*, increasing the number of clients)
- Synthetic benchmarks may need to support multiple configurations to mimic distinct behaviors found in real production settings
  - **Interfaces** (*e.g.*, CPU, memory, storage, network APIs)
  - **Operations** (*e.g.,* load, store, read, write, stat, mkdir, send, receive, …)
  - **Access patterns** (e.g., sequential, random, Zipfian, …)
  - But that's not all…

Workload

Synthetic workloads

- There are other properties of the workload that may need to be configured carefully to reproduce realistic evaluation scenarios
  - Operation **arguments** (*e.g.*, size/duration of requests)
  - **Parallelism** (e.g*.,* number of processes, threads)
  - Workloads with **mixed operations** (*e.g.*, reads vs writes, sends vs receives)
  - **Rate** of client requests (*i.e.*, stress testing vs controlled client throughput)
- And even more tricky characteristics, for instance
  - Temporal and spatial **locality** of requests (*e.g.*, important for assessing memory and storage accesses)

Synthetic Benchmarks

Some examples

- **OSBench**: Used for creating processes, memory allocation, creating files, …
  - https://gitlab.com/mbitsnbites/osbench
- **Stress-ng**: Allows benchmarking CPU, RAM, network, disk, …
  - https://github.com/ColinIanKing/stress-ng
- **Sysbench**: Allows evaluating CPU, RAM, I/O (file system, databases)
  - https://github.com/akopytov/sysbench
- **FIO/DEDISbench**: Used for testing **block/file-based data I/O operations**
  - https://github.com/axboe/fio, https://github.com/jtpaulo/dedisbench
- **Filebench**: Used for evaluating **file system data and metadata I/O operations**
  - https://github.com/filebench/filebench
- and many more…

Environment

Hardware and software

- Knowing and setting up the right environment to run experiments is very important!
  - Testing your SUT in an unrealistic environment may lead to **wrong conclusions**
  - It is also important for others to be able to **reproduce** your results
- One must be able to **characterize the experimental environment**
- **Hardware**
  - What CPU, RAM, GPU, network, and disk models are being used?
  - What hardware configurations are being used? (e.g., number of CPUs, amount of RAM, …)
- **Software**
  - What is the version of the SUT, its components, and the underlying software stack?
  - For instance, if one is evaluating a file system, what version is it? What operating system is being used? And the kernel version?
  - What about used libraries and corresponding versions?
  - Are there any special configurations being tested?

Metrics

Understanding your SUT

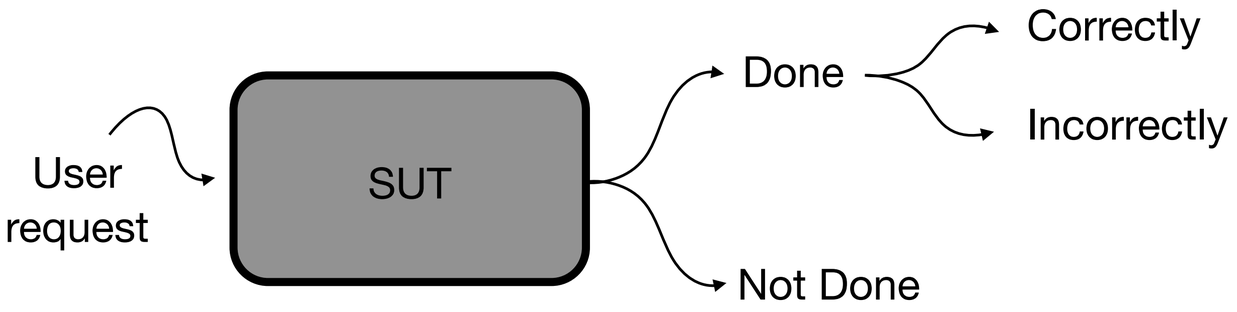
- **Performance** of the system
  (*e.g.*, latency, throughput of operations[1], hits vs misses in a cache)
- **Utilization** of resources
  (e.g., CPU, RAM, network, disk, energy consumption)
- **Efficiency** of the system
  (e.g., the ratio between performance and resource usage)
- **Reliability** and **Availability** of a system
  (e.g., number of errors, failures, uptime)
- **Fairness**, **accuracy**, …

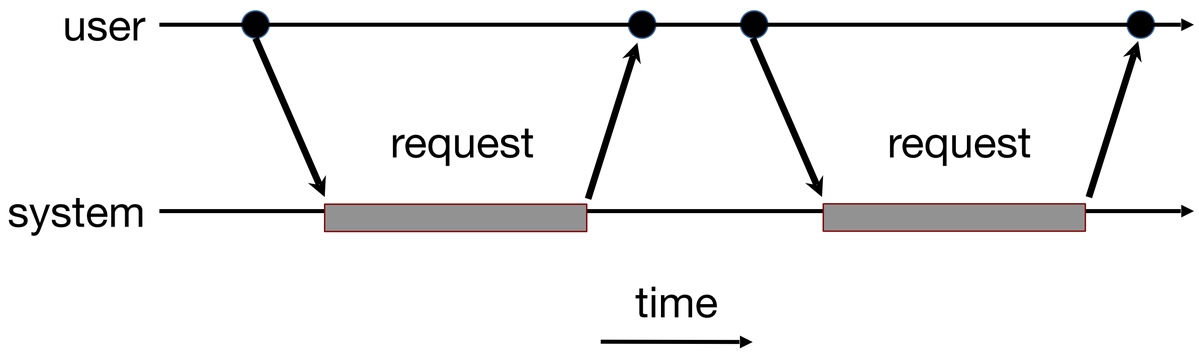
[1]A naive view (latency = 1 / throughput) is false! This is only true when the system is busy 100% of the time executing exactly 1 request

Performance

Requests

- The SUT will be serving **multiple user requests over time**
- The **response** varies
  - Some requests are served **correctly** or **incorrectly** (e.g., errors)
  - Other requests may **not be served**
    (e.g., the SUT rejects a request because it is too busy)

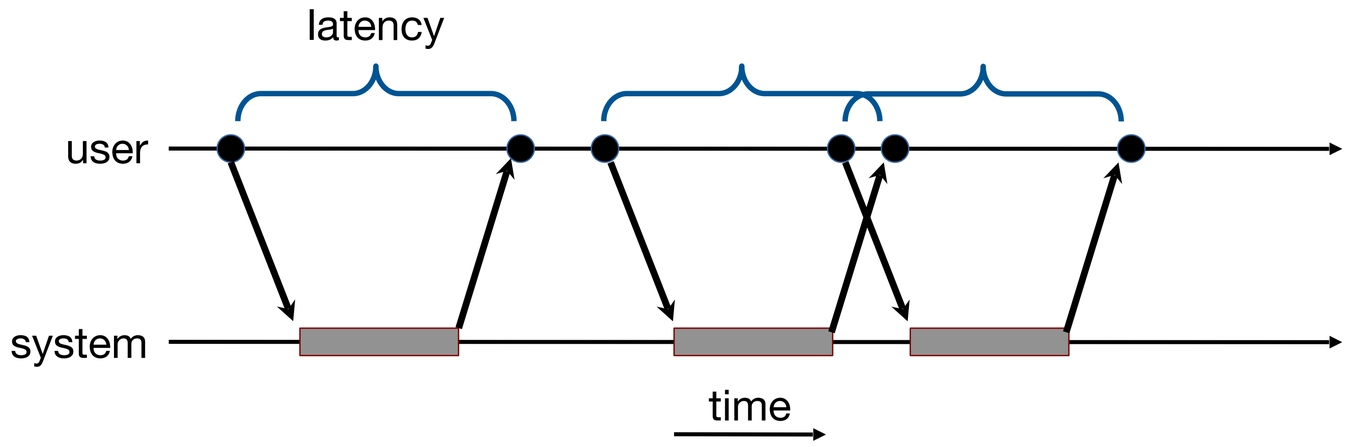


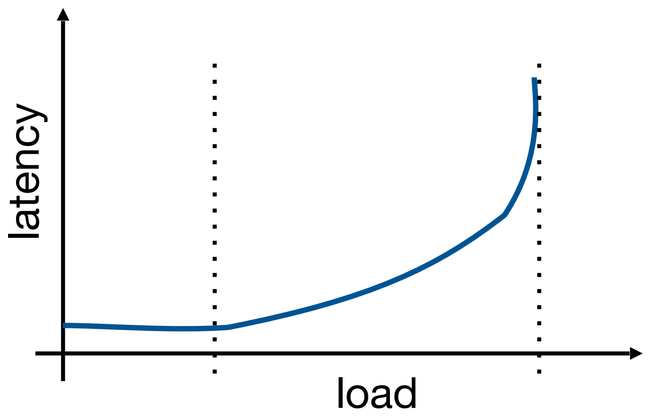


Performance

Metric: Latency

- **Latency:** The time interval between the user's request and the system's response
  - Also referred to as **response time**
- As the **load** in the system **increases**, the **latency** of requests tends also to **increase**

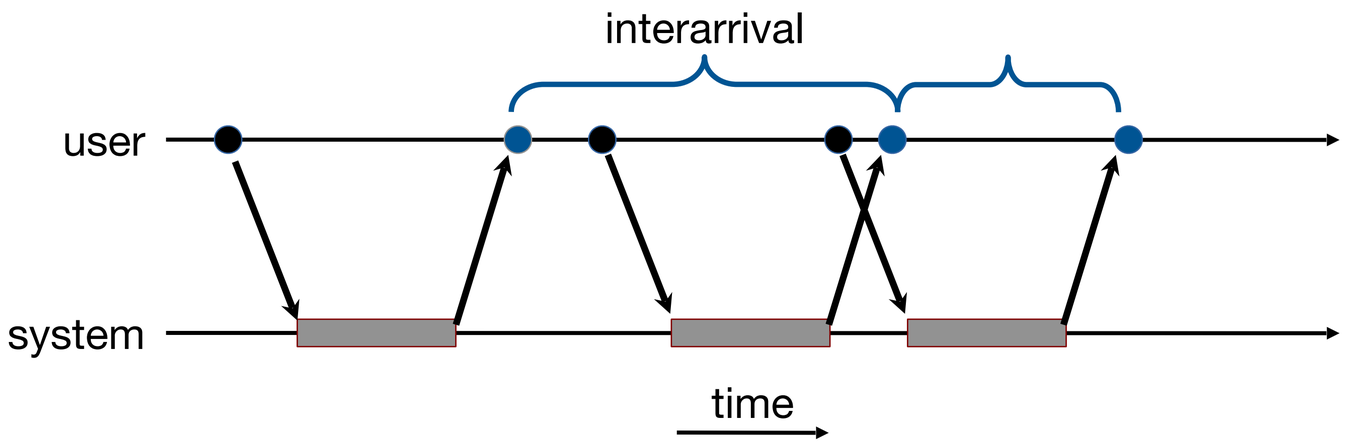


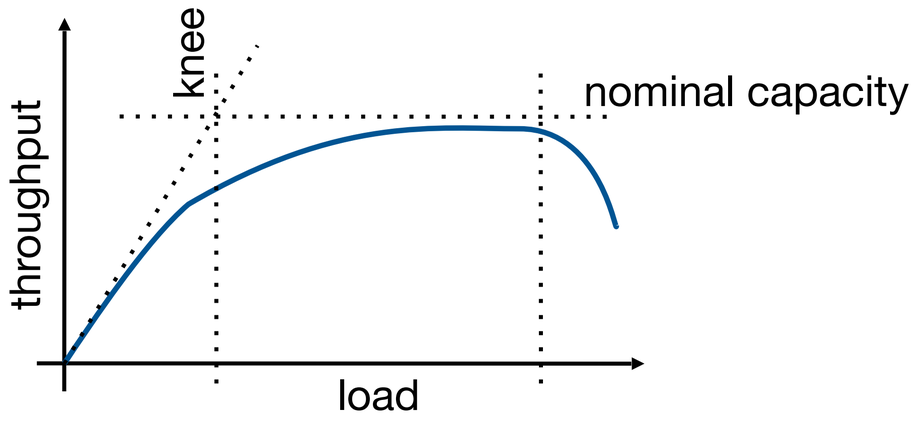


Performance

Metric: Throughput

- **Throughput:** rate at which user requests are serviced by the system (i.e., operations served per unit of time)
- As the **load** in the system **increases**, the **throughput** of requests tends to increase until a certain point, reaching the system's **nominal capacity**
  - When the system becomes **saturated**, the throughput starts decreasing





Response Time vs Throughput

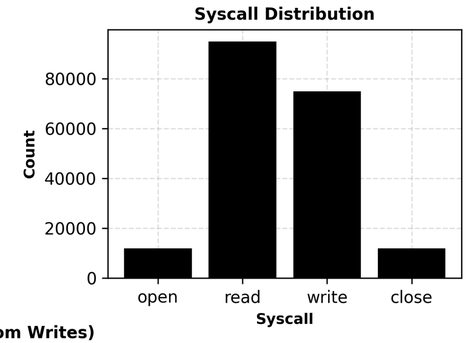
Carefull!

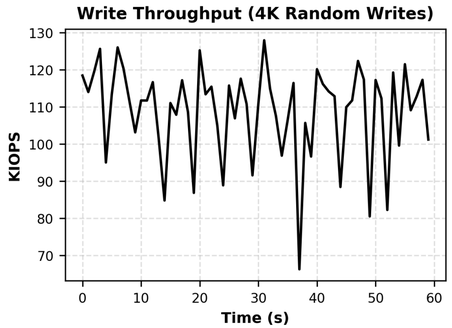
- A naive view (**latency = 1 / throughput**) is false!
  - This is true only when the system is 100% busy, executing exactly 1 request at a time!
- Ideally, one would **optimize** a system to increase throughput and decrease latency
  - Hard to achieve!
- Most optimizations **trade** one metric for the other
  (e.g., batching requests usually improve throughput at the cost of latency)

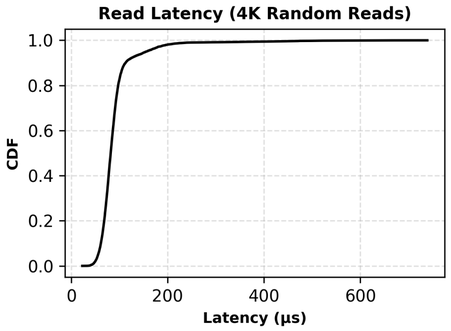
Analysis and Representation

Of collected metrics

- When running experiments, one must collect several **samples** for metrics of interest!
  - Repeat your experiments several times to identify **abnormal behavior** (outliers)
  - Remember that many workloads **may not be deterministic**!
  - The behavior of the SUT and the underlying software stack (e.g., the OS) may not be deterministic either!
- After collecting the samples, these must be **analyzed** and, ideally, **summarized**
  - Mean, standard deviation (std. dev.), percentiles, confidence intervals, …
- Use **visual representations** to observe and better understand your samples!
  - Time-series, histograms, Empirical Cumulative Distribution Function (ECDFs), …







Useful tools

Metric collection and analysis

- Metric collection
  - Most benchmarks already report some metrics (*e.g.*, throughput, latency, …)
  - Use tools, such as *dstat*, *iostat*, *pidstat* … for OS resource monitoring
    - Or more advanced ones (*e.g.*, *Beats* + *ElasticSearch*, *Prometheus*)
- Analysis
  - Scripts! (*Python* is nice for this…)
  - *Gnuplot* for plots!
  - Or more advanced visualization tools such as *Kibana* and *Graphana*

Common Mistakes

When benchmarking storage systems

- No goals or biased ones
  - Define the **evaluation questions** you want to answer with your experiments!
  - First, ask the questions…
- Unrepresentative benchmarks
  - Choose **realistic workloads**, **metrics,** and experiments that help answer your questions
  - Avoid bias by only showing workloads, environments, and metrics that perfectly fit your solution. Don't be afraid to show **strengths** and **drawbacks!**
- Compare your solution with other ones
  - For example, with **baseline setup**(s) to show the benefits/drawbacks of your design and optimizations
  - Or other **related solutions** (i.e., the state of the art!)
  - Again, **be fair** in your comparison!

Common Mistakes

When benchmarking storage systems

- Unsystematic approach - **Reproducibility is key!**
  - Set and document your workloads, experimental environment, and configurations!
  - Be sure that your environment is set the same way for each experiment
- Software and hardware may exhibit distinct behavior across experiments
  - For instance, due to caches[1], background OS tasks, …
- Wrong analysis and presentation of results (i.e., misunderstanding measurements)
  - **Inspect samples in time** for stability
    - Consider, again, external phenomena (e.g., caches, background tasks)
  - Use appropriate **summarization techniques** and **visual representations**

[1] The OS page cache, for instance, may be dropped before running each experiment for increased reproducibility across experiments

More Information

- R. Jain. ***The Art of Computer Systems Performance Analysis.*** Wiley, 1991.
  - Chapters 1 to 6 and 9 to 13
- Gernot Heiser. ***Systems Benchmarking Crimes*.**
  https://gernot-heiser.org/benchmarking-crimes.html
- Tarasov et all. ***Benchmarking File System Benchmarking: It *IS* Rocket Science.***
  HotOS, 2011.

Questions?